Online Learning: Foundations and Applications in Classification and Time Series Forecasting

Erik Eremenko and Sam Muller

Abstract

Machine learning systems deployed in real-world environments must operate under changing data distributions, yet most traditional models are trained offline on fixed datasets and lack the ability to adapt after deployment. Online Learning (OL), which includes incremental and continual learning paradigms, addresses this limitation by enabling models to update continuously from streaming data while preserving performance on previously learned tasks. However, OL introduces significant challenges, particularly catastrophic forgetting, computational constraints, and the need to make predictions without access to future samples or repeated passes over past data. This paper provides an overview of the foundations of OL through regret minimization, presents key challenges that arise when learning from data streams, and surveys practical methods designed to mitigate catastrophic forgetting, including regularization-based approaches, replay mechanisms, task-specific components, and knowledge-distillation strategies. We additionally highlight applications in classification, regression, and large-scale structure learning, with special emphasis on remote sensing and time-series forecasting, where OL offers substantial advantages in adaptability and memory efficiency. By synthesizing theoretical and practical advances, this work illustrates how OL methods enable machine learning models to remain robust and responsive in dynamic, non-stationary environments.

Index Terms

Online Learning, Incremental Learning, Continual Learning, Classification, Regression, Time Series Forecasting, Convex Optimization, Catastrophic Forgetting, Regret Minimization

I. Introduction

TRADITIONAL machine learning methods and deep learning approaches have typically learned from a closed set of training samples. Provided multiple attempts to learn from the dataset, the models arrive at parameters that minimize their loss function, representing that model's best attempt at learning the underlying data distribution of the problem at hand. The assumption in this approach is that the distribution of the observed training samples is static. In reality, our environments are dynamic and sampling from these environments will produce varying data distributions at different times.

Two complementary paradigms have emerged to enhance the adaptability of machine learning models to dynamic environments. *Incremental learning* generally investigates how new information can be integrated from an incoming data stream without overwriting existing knowledge, whereas *online learning* examines how models can update continuously from a stream of incoming data, often one sample at a time. Both paradigms belong to the broader domain of *continual learning*. In continual learning approaches there is often no prior knowledge of the distributions of the predictor variables [1], [2].

Models that incorporate characteristics of both online and incremental learning are known as *Online Incremental Learning (OIL)* models [2]. For simplicity, we will refine this label to Online Learning (OL). The central objective of OIL is to preserve adaptability to non-stationary environments. To achieve this, OL models depart from the conventional batch-learning paradigm and instead update continuously as new data arrive in a data stream.

A *data stream* is an ordered sequence of samples generated in real-time. The form of the data sample can be of any form (i.e., a number, vector, matrix, image, graph, word, etc.) [3]. Data streams can be (and usually are) non-stationary. Additionally, the technology producing the data stream might evolve with time.

Online Learning models introduce new computational and performance challenges due to their continual learning approach. This report will discuss these challenges and explore novel methods developed to address them. In addition, it will examine the advantages and practical applicability of OL to highlight its potential and to motivate solutions to the difficulties inherent in this approach.

A. Importance

The constant development and use of new technologies by consumers and industry now produce vast amounts of data in real-time, bringing society into a new era characterized by this constant stream of *big data*. There is valuable knowledge to learn from this data, however, the "flood of big data has outpaced our capability to process, analyze, store, and understand these datasets" [3]. New algorithms are needed to ensure the efficient use of training resources while keeping models adaptable to changing environments.

The need for OL models is further motivated by the common need to retrain a model after its performance degrades with time. An offline model's prediction accuracy can degrade due to concept drift, changes in data quality and labeling consistency,

Thanks to Iryna Burak for her support

changes in the sensors technology, out of date model representations (i.e., not incorporating new vocabulary instances), and feedback effects where the predictions of a model influence its subsequent observations [2]. These unpredictable causes of model degradation can be mitigated through the use of the OL paradigm, where models can continuously learn from a stream of data influenced by changing data distributions and sensor technology [2].

A practical real-world example where OL methods should be used is the case of an object detection model for self-driving cars. At the time of deployment the model has likely been trained to detect pedestrians, cars, cyclists, traffic signs, sidewalks and so on. However, our cities are dynamic environments and it is possible that the car's object detection model has never seen an e-scooter, for example. It is therefore necessary for the model to incrementally learn from these new observations while maintaining its ability to detect previously learned classes. In fact, OL is necessary to build and maintain self driving car systems that are capable of identifying all the cars on the road, especially as new types of cars are released every year. As the typical appearance of a car changes, we can think of this as an evolving probability distribution over all the possible car designs.

To further motivate the OL paradigm, in many ways OL is more similar to the learning method used by humans than offline batch learning. Humans are not given a set amount of time to repetitively study a set of samples meant to represent the world before they enter it. They learn in real-time with a constant stream of experience, with no opportunity to go backwards to try again. We only have the ability to learn and adapt to the data stream. Furthermore, we do not re-train ourselves on replicated moments from the past (like Neo in the Matrix), we are able to encode and store our experiences as memories (representations). As such, developing and training models in line with the OL paradigm might bring our models closer to the plasticity and adaptability that have been so important in the success of human civilization. In fact, some OL methods explored in this paper are inspired by mechanisms within the human brain [4], [5]

B. Definitions

A data stream $\mathcal{D}$ can be defined as a sequence of observations $\{x_i\}_{i=1}^{n}$. The sequence continues to grow as more observations are made. The probability density function (PDF) that describes the likelihood of observing $x \in \mathbb{R}^d$ is given as $p_X(x)$, and is often assumed to be stationary, meaning its statistical properties do not change with time. In the setting of a dynamic environment, the underlying distribution of x can change with time and is more accurately represented as a function of time $p_X(x,t)$. The change in statistical properties of the underlying data distribution over time is called *concept drift*.

To understand how OL models work, we first have to introduce the underlying concept of *regret minimization*. Formally, the framework consists of a learner whose task it is to predict the solution $y_t \in Y$ of T rounds of instances $x_t \in X$. The observations take the form of a data stream $\mathcal{D}$ and are supplied by an environment that may be adversarial or stochastic. As the learner determines its prediction $p_t \in P$, the environment reveals the solution y_t and the learner suffers a loss $\mathcal{L}_t(p_t, y_t)$. This loss should be minimized during the learning process. In order to define this minimization goal, the concept of *regret* is introduced, which describes how much the predictions of the algorithm of the learner deviate from the best single hypothesis in hindsight $h \in H$ over the course of T rounds [6].

$$Regret_T(h) := \sum_{t=1}^{T} \mathcal{L}_t(p_t, y_t) - \min_{h \in H} \sum_{t=1}^{T} \mathcal{L}_t(h(x_t), y_t)$$

In general, an *OL* algorithm is deemed to be successful if it can guarantee that the average $Regret$ grows sublinearly with T, e.g. $Regret_T \in o(T)$, which implies that in hindsight, the algorithm performs on average as well as the best fixed strategy and that the average regret per round t goes to zero for large T [7]. Typical regret rates are $\mathcal{O}(\sqrt{T})$ for convex Lipschitz functions [6, ch. 2.4] and $\mathcal{O}(\log T)$ under strong convexity [8, ch. 4].

A large class of computationally efficient *OL* algorithms arises from the domain of convex optimization. As Boyd and Vandenberghe emphasize in [9], a surprisingly wide range of problems allow for reformulation as convex optimization problems. The property of guaranteed efficiency of the algorithms based on convex optimization and the vast amounts of problems favorable to *convexification* have made this framework central for efficient online learning.

One such classical example is *online linear regression*, which can be formulated as a convex optimization problem as follows: minimize the convex loss function $\mathcal{L}_t(w) = |\langle w, x_t \rangle - y_t|$ with the prediction $p_t = \langle w_t, x_t \rangle$. Here, $w \in \mathbb{R}^d$ represents the learnable parameters (weights) that are iteratively updated by the algorithm and x_t is the feature vector at time step t. Another fundamental algorithm is Online Gradient Descent (OGD), which extends the standard gradient descent to the online setting. It works by updating parameters using only first-order gradient information: $w_{t+1} = \Pi_W(w_t - \eta_t \nabla_t \mathcal{L}_t(w_t))$, where η is the learning rate and Π_W expresses the Euclidean projection onto the convex set W. On the other hand, various online learning problems require more sophisticated convexification procedures, as demonstrated in [6]. Some approaches rely on surrogate loss functions or randomization. Other approaches, such as the Follow-the-Leader framework, exploit specific structural properties of the loss sequence.

While general convexity is sufficient for algorithms like OGD, there are problems that allow for faster learning by exploiting the properties of their specific Loss functions. Specifically, approaches relying on exp-concavity can achieve tighter regret bounds. This is useful in online learning because exp-concavity allows algorithms such as Online Newton Step (ONS) to

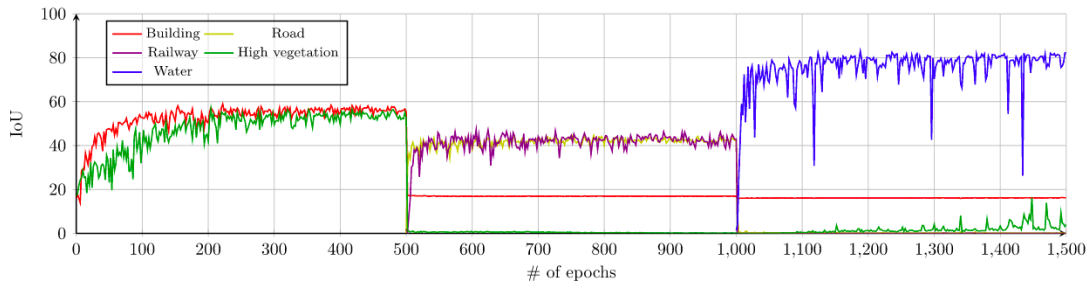

Fig. 1. Figure 7d from [11] demonstrating the loss of accuracy (IoU) on previously learned classes as the model is fine-tuned on new classes.

obtain logarithmic regret $\mathcal{O}(\log T)$, so the learner approaches the best fixed predictor in hindsight much faster than with general convex losses (typically $\mathcal{O}(\sqrt{T})$). Formally, a loss function $\mathcal{L}$ is α-exp-concave if $e^{-\alpha\mathcal{L}(x)}$ is concave for some $\alpha > 0$ [8]. This definition implies a lower bound on curvature as the Hessian satisfies $\nabla^2\mathcal{L}(x) \succeq \alpha\nabla\mathcal{L}(x)\nabla\mathcal{L}(x)^\top$ as shown in [10, Lemma 4.2] where $\succeq$ denotes the Loewner order (meaning that the difference between the matrices is positive semidefinite). This relationship means that the curvature and thus the Hessian is large in the direction of the gradient, a fact that can be used by algorithms like Online Newton Step (ONS) to approximate the geometry of the loss function using only first-order information and achieve logarithmic regret, significantly improving over square-root regret of general convex functions [10, p. 4.4]. The ONS algorithm constructs a positive semidefinite matrix A_t as a surrogate for the Hessian matrix to approximate the curvature, defined as $A_t = \sum_{s=1}^{t} \nabla_s \nabla_s^\top + \epsilon I$ which is then used in the quasi Newton update rule $w_{t+1} = \Pi_W^{A_t}(w_t - \frac{1}{\gamma}A_t^{-1}\nabla\mathcal{L}_t(w_t))$ where $\Pi_W^{A_t}$ expresses the projection onto W with the norm induced by A_t.

II. Challenges

It has proven to be a challenge in the online learning community to train models that learn from new observations while continuing to perform well on past observations. This phenomenon is known as *catastrophic forgetting*. The most naive approach to online learning from a stream of data could be thought of as simply allowing the model to continually optimize its parameters as new samples arrive without any special algorithms or approaches. Tasar, Tarabalka, and Alliez [11] name this simple approach as *fine tuning*. Figure 1 shows a classic example of catastrophic forgetting [11] where the model loses its ability to classify buildings and high vegetation after the task of classifying railway areas and roads are introduced.

A number of methods have been proposed to combat the occurrence of catastrophic forgetting, including regularization approaches [12], [13], the use of generative models [4], storing exemplar samples [14], the use of task-specific components [5], and knowledge-distillation strategies [15]. The possible methods to mitigate catastrophic forgetting will be addressed in Section III-B.

In the context of online regression the avoidance of catastrophic forgetting is related to regret minimization. A model with high regret likely performs well on the local part of the time-series it has recently been trained on, but has lost its accuracy for earlier parts of the time-series. In other words, a model with high regret has catastrophically forgotten its learnings from earlier sections of a time-series.

The next sections will explore how OL applies to the settings of classification and regression.

III. Classification

A classification model that can continually learn online will perform better in a changing world. For example, imagine if the computer vision models used for self driving cars did all of their learning before the popular use of shareable e-scooters. These models would have no concept of these scooters and would fail to properly identify them. In the real-world, new classes can arise that are important for our models to learn *while* they maintain their ability to recognize previous classes.

A. Scenarios for Online Classification

For a given data stream $\mathcal{D} = \{x_i\}_{i=1}^n$, we can consider corresponding labels y_i for the classification task. Comparing the performance of different online learning algorithms is dependent on whether prior information is available about the task being performed. Ven and Tolias [16] propose three distinct settings for classification problem in online learning in order to help standardize the comparison of online learning algorithms.

*1) **Task Incremental Learning**:* Samples from the data stream arrive at the time of inference with knowledge of the task they are expected to perform and take the form: $\{(x_i, k_i)\}_{i=1}^n$. An example of this scenario is training a image classifier to classify apples as ripe or unripe. The model might be useful to both farmer and home chefs, so it is trained on images from outdoors on farms and in kitchens. In Task Incremental Learning (TIL) it will be known whether the input is from outdoors or indoors, so a terminal classification head H_i can be used for each photo setting (k_i), but have some shared hidden layers for extracting useful features. Given knowledge of the task k_i, it is only necessary to look at the output from H_i to achieve our prediction of ripe or unripe. In a TIL setting it is common to make use of multiple classifiers and it is generally the easiest setting to work with.

*2) **Domain Incremental Learning**:* In this setting the task ID k_i is unknown at the time of inference, however the desired prediction remains the same. This means that for our apple ripeness predictor we have no means to distinguish between outdoor and indoor photos. Although the task is unchanged (ripe vs. unripe), the input distribution changes, and the model must adapt to the new domain while retaining performance on earlier examples.

*3) **Class Incremental Learning**:* In this setting the task ID k_i is unknown at the time of inference *and* the model must distinguish between *different* tasks on its own. For example, this means extending the apple ripeness predictor to other fruits. Initially the model had only two outputs, ripe or unripe apple. As new fruits enter the data stream, additional outputs are added to the classification head (ie. the output vector dimension is increased) and consequently new parameters are added to the model. For example after learning from a data stream containing apples, bananas and avocados the necessary output would be:

$$f(x) = \quad \hat{\mathbf{y}} \quad = \begin{bmatrix} y_{\text{apple, ripe}} \\ y_{\text{apple, unripe}} \\ y_{\text{banana, ripe}} \\ y_{\text{banana, unripe}} \\ y_{\text{avocado, ripe}} \\ y_{\text{avocado, unripe}} \end{bmatrix}$$

The previously mentioned settings of Task, Domain and Class Incremental Learning all assume that the task is known during *training*. Zeno, Golan, Hoffer, *et al.* [17] extend these three online classification settings to include two previously unaddressed instances where the task is **not** known during training.

Expanding Scenario Definitions: Zeno, Golan, Hoffer, *et al.* [17] propose a fine-grained categorization of continual learning scenarios that generalizes the previously defined Task-, Domain-, and Class-Incremental Learning (TIL, DIL, CIL). They propose the concept of *task-agnostic* settings, where the boundary between tasks in the data stream is not known. This can occur due to a gradual introduction of new task types in the data stream over time, or due to a continuous change of the underlying distribution that the data stream samples are taken from.

In both cases, the model receives no explicit signal of task changes. Consequently, continual learning must proceed in a fully *unsupervised task regime*, where the algorithm has no knowledge of the underlying distribution of tasks or when new ones emerge. Zeno, Golan, Hoffer, *et al.* [17] refer to such scenarios collectively as Task-Agnostic Continual Learning (TACL).

These TACL scenarios can be situated in the context of the fruit ripeness model. In a task-agnostic *domain* incremental setting, photos might gradually shift from outdoor to indoor environments, altering illumination and texture statistics without any clear task boundary or knowledge of the shift in conditions. The primary difference that this would cause is that the model does not first optimize its weights for indoor settings and then alter the weights to also perform on outdoor settings. The optimization is constantly responding to both indoor and outdoor settings. In a task-agnostic *class* incremental setting, the data stream may initially contain only apples, but over time, images of bananas and avocados begin to appear without any explicit indication that new classes have been introduced. So, similar to task-agnostic *domain* incremental learning the model will alternate adjusting its parameters for different tasks.

These task-agnostic settings reflect a more realistic view of how models encounter data in the wild, where changes are gradual, overlapping, and unlabeled by task identity.

The next subsections will explore some practical methods that can be applied depending on the classification scenario (i.e TIL, DIL, CIL).

B. Practical Methods

Task Specific Components

These methods are constrained to the setting of Task Incremental Learning, where knowledge of the task being performed is given provided at the time of inference. With knowledge of the task being performed, particular models or sections of the model can be selected to perform the task. The use of **Multiple Models** is the simplest approach. A separate model is trained or each task and at the time of inference only the corresponding models makes a prediction.

Masse, Grant, and Freedman [5] proposed the neurologically inspired **Context Dependent Gating (XdG)** method to allow different regions of one model to learn multiple tasks. XdG uses X as a hyperparameter. $X\%$ represents the fraction of learnable parameters set to 0 (gated) for each task. Masse, Grant, and Freedman [5] found $X = 87\%$ to be an effective fraction. In doing this, the remaining $(100 - X)\%$ of parameters are learnable for the new task.

It was found that XdG alone was unable to effectively learn new tasks while retaining performance on old tasks. Although only $(1 - X)\%$ of parameters would change with each task, some of these parameters are shared by other tasks and changing them significantly increases the likelihood of catastrophic forgetting. The best results were found when XdG was combined with a regularization method such as SI or EWC which penalize changes to model parameters that are important to other tasks. [5].

Regularization

With this approach a regularization term is added to the loss to penalize changes to parameters that were important in performing former tasks. This prevents freely updating weights (open fine-tuning [15]) which then might result in catastrophic forgetting of past tasks.

$$\mathcal{L}_{\text{total}} = \lambda\, \mathcal{L}(Y, \hat{Y}) + (1 - \lambda)\mathcal{R}(\theta)$$

Where λ is a hyperparameter that balances the loss due to regularization. Ven and Tolias [16] discuss two specific methods of formulating the regularization function. With **Synaptic Intelligence (SI)** [12], the total contribution to reducing the loss function is tracked for each parameter. Using this metric the regularization term will penalize changes to important parameters. **Elastic Weight Consolidation (EWC)** [13] uses the Fischer Information for each parameter to determine the sensitivity of the output to perturbations of that parameter. The regularization term penalizes changes to parameters that have a strong influence on the model's output. Both SI and EWC aim to balance a model's memory of old tasks with its adaptability to new tasks.

Modifying Training Data

Additional input-output pairs $(x_i, \hat{y}_i)$ can be added to the training process to maintain performance on past tasks. Depending on the memory constraints of the online learning system, some samples or even models might be stored to incrementally 'remind' the model how to perform on the old tasks.

The **Exemplar Method** as described by [14] stores an exemplar sample of each class to mix in to the training sequence of new tasks. An exemplar can be chosen as

$$x_p^* = \arg\min_{x_i \in \mathcal{X}_p} \left\| x_i - \frac{1}{|\mathcal{X}_p|} \sum_{x_j \in \mathcal{X}_p} x_j \right\|_2$$

Where $\mathcal{X}_p$ is the set of all feature vectors x_j belonging to the class p. This minimization creates an exemplar pair (x_p^*, y_p) that represents the sample input x_i that is closest to the center of the cluster of samples in the feature vector space.

Deep Generative Replay (DGR): Alternatively, a generative model can generate pseudo-samples from earlier tasks to help the model avoid forgetting them, as suggested in the Deep Generative Replay method [4]. In this approach the overall model is described as the Scholar and is composed of a Generator model G and a Solver model S. As its name suggests, the Generator uses an adversarial framework to generate novel samples from the distribution $p_x^i(x)$, the probability density function over the samples of task k_i. As samples conforming to task k_i arrive from the data stream the Solver learns from the training tuple (x, y) and the Generator learns to generate sample samples x_i where i is the ith task. Once new tasks k_{i+1} begin to arrive in the data stream the former Generator is labeled as G_{old} and the Solver as S_{old}. A new Generator is initialized and a new Solver is initialized with the weights of S_{old}. The training of S_{new} subsequently consists of data (x_{i+1}, y_{i+1}) from the new task k_{i+1}

as well as generated data (x_i, y_i) to represent 'memories' of past tasks. The corresponding label y_i for the generated sample x_i is created by passing x_i through S_{old}.

With this framework, during training 4 models must be stored, 2 Generators and 2 Solvers. Once training has concluded, the solver S should approximate the inference operators for all tasks in the collection

$$\mathcal{K} = \{\, k_i \mid i = 0, \ldots, n \,\}.$$

The DGR algorithm, as presented in [4], assumes that tasks in the data stream have distinct divisions and that it is known when the task changes.

Learning Without Forgetting (LwF) is a method that aims to maintain a model's *response* to old tasks as new tasks are introduced in the data stream [15]. In a manner similar to *transfer learning*, two models are used: a *teacher* model trained on the previous tasks that will "teach" the old tasks to the newly initialized *student* model. Essentially, when a new task arises in the data stream, a new model $f_{new}(x)$ is created by extending the output vector and adding and initializing new model parameters necessary for the extended output. Therefore, $f_{new}(x)$ is composed of shared parameters for hidden representations, old parameters for old tasks, and new parameters for the new task $(\theta_s, \theta_o, \theta_n)$. The response of the teacher model $f_{old}(x)$ is measured for each new training pair (x_i, y_i) and is used as a *soft target* for the new model $f_{new}(x)$ to learn from. A new loss term $\mathcal{L}_{\text{old}}(Y_o, \hat{Y}_o)$ is added to the loss equation, representing the distilled "knowledge" of the teacher model. Minimizing this loss motivates the parameter update to maintain a similar response to former tasks.

The loss function for LwF can be summarized as follows:

$$\mathcal{L}_{\text{total}} = \lambda_o \, \mathcal{L}_{\text{old}}(Y_o, \hat{Y}_o) + \mathcal{L}_{\text{new}}(Y_n, \hat{Y}_n)$$
$$\theta^* = \arg\min_{\theta} \Big(\mathcal{L}_{\text{total}} \Big)$$

Where $\hat{Y}_o$ represents the set of soft target outputs generated by the teacher model and each of the output heads for the previously learned tasks. $\hat{Y}_n$ are the one-hot encoded output vectors for the new tasks, and λ_o balances the influence of the distillation loss $\mathcal{L}_{\text{old}}$ and the new task loss.

A Temperature parameter T_temp can be used when computing the soft max targets to preserve broader knowledge of the class similarities learned by the teacher model. Typically $T_temp > 1$ provides a 'softer' softmax.

$$\hat{y}_i^{'} = \frac{\hat{y}_i^{1/T_temp}}{\sum_{j=0} \hat{y}_j^{1/T_temp}}$$

For example a soft target output of $[0.93, 0.01, 0.03, 0.03]$ will be softened to an output of $[0.69, 0.07, 0.12, 0.12]$ when a temperature $T_temp = 2$ is used.

Learning Without Forgetting was shown by [16] to perform better than simply adding a regularization term to penalize deviation from the previous model parameters. However, the time it takes to use LwF to train a model on a new task will increase linearly with the number of former tasks in the data stream because the new task k_i must be run through all previous output heads $H_j : j = 0, 1, ...(i-1)$.

C. Application: Remote Sensing

Remote sensing is a field that uses various sensors (optical, RADAR, LiDAR etc.) to quantify properties of the Earth's surface. The sensors can be placed on satellites, airplanes, Unpiloted Aerial Vehicles (UAVs) or even a person working in the field. Each sensor is characterized by a spatial resolution, a resolution in the parameter it measures (e.g a camera samples the visible light spectrum, while a radar antenna samples the microwave domain of the light spectrum), a temporal sampling frequency (for satellites) and sources of noise. Due to their continual measurement with each orbit, the observations of Earth made by satellites produce a massive data stream

Humanity continues to add new satellites to Earth's orbit, generating vast amounts of time-series data in the form of satellite imagery. The European Union's Copernicus satellite program alone generates 20 Terabytes of new imagery every day [18]. Storing all of this imagery for a machine learning model to iteratively learn from is not technically feasible, demonstrating the value that OL models can bring to the field of remote sensing. Depending on the selected method, as mentioned in III-B, the memory footprint of an OL model would only be the memory required for the model itself and some memory for the use of additional models or exemplar samples, instead of the many terabytes necessary to train a model in the typical offline manner.

Tasar, Tarabalka, and Alliez [11] explore how online learning algorithms can be used to continually teach a Convolutional Neural Network (CNN) to semantically segment satellite images of urban landscapes as the data stream of images progressively introduces new possible pixel classes. Figure 2 shows an overview of incrementally teaching a CNN 4 distinct land types.

The method used by Tasar, Tarabalka, and Alliez [11] is algorithmically the same as Learning Without Forgetting [15]. Before modifying the past model trained on classes $C = \{C_1, C_2, ...C_{n-1}\}$ to learn class C_n, the input x_n is passed through the previous model to generate an output vector of *soft targets*. The authors then use the deviation of the new model from

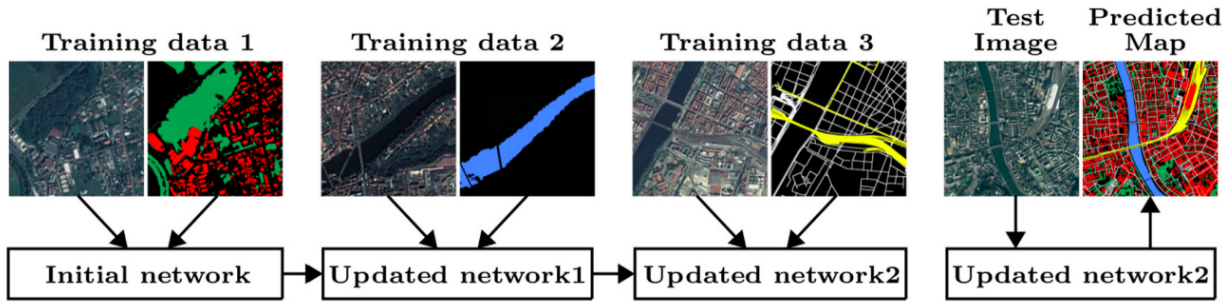

Fig. 2. The progressive introduction of new classes into the data stream and the incremental re-training of the network [11]. Pixel Legend: green: Vegetation, red: Urban Buildings, blue: Water, yellow: Railways. (Far Right) Running inference on a test image using all learned classes.

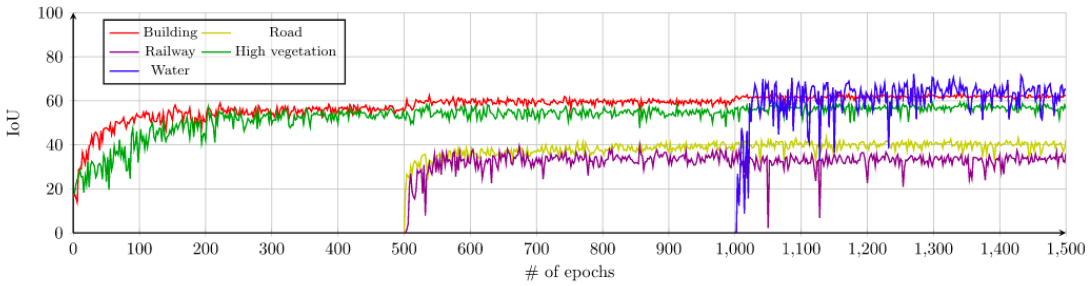

Fig. 3. Incremental Learning algorithm successfully 'remembering' past classes as new classes are introduced.

these soft targets as a *distillation loss* $\mathcal{L}_{\text{distillation}}$, ensuring that the new model's response does not deviate significantly with the belief that this maintains performance on past classes.

The authors compare the performance of their algorithm against several other common methods of incorporating new observations into a model. Figure 1 shows the naive use of the method called *fine-tuning* where a single model continually has all of its parameters optimized as new classes arise. In contrast to Figure 1, Figure 3 demonstrates that transferring distilled knowledge using soft targets within the teacher–student paradigm helps to preserve prior knowledge and avoid catastrophic forgetting.

The application of online learning to remote sensing data naturally extends to the challenge of building sensor-agnostic models. As new satellites are launched, with sensors that measure different spectral bands, resolutions, or noise characteristics, a model trained on one dataset may fail to generalize to the data streams produced by these newer instruments. Instead of viewing the system as a single static model that learns once and predicts indefinitely, we can shift to a perspective in which a lineage of models continuously learns from evolving data streams and passes on its acquired knowledge as new datasets emerge. This can be implemented, for example, through student–teacher paradigms or other continual learning frameworks. Such a perspective aligns well with the central tenets of online learning: enabling models to continually update while retaining essential knowledge from previous tasks, thereby supporting long-term adaptability across heterogeneous remote sensing sensors.

IV. Online Time Series Forecasting

A. Application: Financial Time Series Forecasting

In a world where data dominates decision-making and data streams of unmanageable sizes are being continuously generated, whether it be from social media, machine sensors or entire financial markets, the challenge lies not in the scarcity of information, but instead in the effort to gain useful insights from this data. As the existing data is mined and analyzed, new patterns may emerge from regime shifts that make the insights from recently analyzed data batches obsolete, as noted by Tajeuna et al. [19]. This is especially true for financial applications such as high-frequency trading, where decisions are made in sub-second intervals and the swift reaction to emerging trends is crucial in order to retain a competitive advantage.

While classical time series models are realized and presented as parametric regression, in OL it is sometimes useful to understand forecasting more generally as sequential prediction, where the learner updates from newly observed data and tries to perform as well as the best predictor in hindsight (regret minimization). Thus, forecasting can refer either to point forecasting, i.e., predicting a continuous-valued X_{t+l}, or to state forecasting, i.e., predicting a discrete event such as the direction of a future price change.

This perspective allows us to connect classical autoregressive approaches (ARMA/ARIMA), regret-minimization online methods, and application-influenced reformulations such as classification-based movement prediction for trading.

Historically, time series analysis emerged with a strong focus on periodicity and seasonality, thus leading to models relying on stationarity assumptions on the forecast parameters, and was subsequently developed into a paradigm with increasingly more flexible tools designed for predictions and analyses on data with fewer theoretical restrictions, making it appealing to a vast amount of domains of application where model adaptability is important, e.g. algorithmic trading and quantitative finance, cybersecurity, and risk analysis. This evolution is evident in the development of increasingly sophisticated models, characterized by the incremental loosening of assumptions imposed on the data. This trend can be seen in the move from models that impose stationarity assumptions on the series, whose statistical properties like mean and variance do not change over time, to more deliberate autoregressive models, which can handle changing parameters by incorporating error terms into a linear combination of the previous series values.

1) Classical baselines: ARMA/ARIMA: To illustrate the evolving assumptions, consider the autoregressive moving average model (ARMA) as presented by Box et al. [20, p. 1.2.1], which represents an important milestone in this progression. In general, the time series is a signal that is observable in discrete, equidistant time intervals, and the task is to forecast the value X_{t+l} of the series using the previous values X_{t-1}, $X_{t-2}, \ldots$ where the forecast is placed for an arbitrary *lead time* l from the current time point t. The lead can be one interval length or multiple: for example, one could use the data from 120 monthly disclosures of revenue figures to place a forecast with a lead time of 1, 2, or even 12 months after the current time point. The ARMA(p, q) model formalizes the approach by expressing the current observation as a linear combination of p past observations (autoregressive component) and q past error terms (moving average component). It thus extends the moving-average (MA) model by adding a linear combination of the previously observed values:

$$X_t = \sum_{i=1}^{p} \phi_i X_{t-i} + \sum_{i=1}^{q} \theta_i \varepsilon_{t-i} + \varepsilon_t$$

where ϕ_i are the autoregressive coefficients, θ_i the moving average coefficients and $\varepsilon_t \sim$ i.i.d. $\mathcal{N}(0, \sigma^2)$ represent the white noise error term. These coefficients are usually estimated from the data using a least-squares approach or maximum likelihood estimation. While the ARMA model produces a point forecast X_t, it can be used to make forecasts for the lead l by successive evaluation of the values until X_{t+l}.

To show the strengths and limitations of the ARMA model, think of a time series analysis of the daily stock price of a company, which is eventually disturbed by a new competitor in the market who challenges the stock's current market domination. The ARMA model would have performed well on the previous trend of the dominating stock but would have had problems with the distributional shift that the regime change would have brought to the market, severely increasing the errors of the forecasts. These shortcomings would be ameliorated to some extent by autoregressive integrated moving average models (ARIMA), but as Liu et al. point out [21], the ARIMA process is still limited by its underlying assumption on the error terms following a white noise process and its adherence to weak stationarity assumptions (covariance stationarity), which limits its application to real-world data. Although ARMA forecasts can be implemented in an online fashion, this autoregressive framework proves too restrictive, and we will thus present alternative OL approaches, where the goal is to minimize the regret in relation to the best predictor in hindsight, rather than to recover the parameters of the underlying distribution, as is the case with the autoregressive approaches.

B. Regret-minimizing online forecasting

1) Benchmark and obstacle: ARMA regret under partial observability: As Anava et al. show in [7], online time series forecasting can be implemented using *improper learning* with *regret minimization* with the assumption that the loss function $\mathcal{L}_t$ is exp-concave. In principle, *improper learning* is defined by a learning approach in which the predictor's hypothesis class

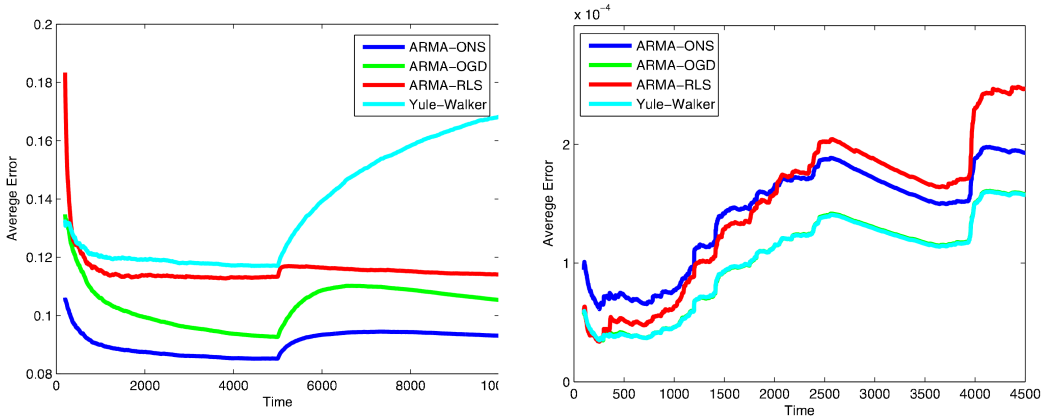

Fig. 4. Comparing algorithm performance. (Left) Cumulative loss under abrupt regime change. (Right) Cumulative loss on S&P 500 daily returns data [7, Fig. 5.1(c), 5.2(b)].

differs from the target class. Let $\tilde{X}_t = \sum_{i=1}^{p} \phi_i X_{t-i} + \sum_{i=1}^{q} \theta_i \varepsilon_{t-i} + \varepsilon_t$ be some ARMA model with coefficients ϕ, θ. With this, we define the regret as follows:

$$Regret_T = \sum_{t=1}^{T} \mathcal{L}_t(X_t, \tilde{X}_t) - \min_{\phi,\theta} \sum_{t=1}^{T} \mathcal{L}_t(X_t, \tilde{X}_t(\phi, \theta))$$

. Hence, the regret is compared to the best possible ARMA forecast in hindsight. However, as the paper points out, this approach is limited by the fact that standard online convex optimization is not applicable in this particular setting. Since the noise terms $\{\varepsilon_t\}_{t=1}^{T}$ cannot be observed by the predictor, the recursive ARMA evaluation relies on unavailable values in its evaluation steps, which makes the regret benchmark of competing against the best ARMA prediction ill-defined, as computing the best ARMA prediction in hindsight requires the knowledge of unobserved noise terms $\{\varepsilon_t\}_{t=1}^{T}$.

2) Improper learning via surrogate AR and ONS: To combat this problem, Anava et al. adopt an *improper learning* approach, in which they decouple the predictions from an ARMA(p, q) model, which mimics the underlying model, and instead define a surrogate autoregressive predictor based on AR(m+k). To achieve an approximation of the ARMA(p, q) predictor with a cumulative loss up to $\mathcal{O}(1)$, m has to be chosen on the order of $q \log T$, as defined in Lemma 3.3 of [7]. This way, the predictor depends solely on the past observed values $X_{t-1}, ...X_{t-(m+k)}$, not on unobservable noise terms. The AR(m+k) predictor with an (m+k)-dimensional coefficient vector $\gamma \in \mathbb{R}^{m+k}$ is formally defined as:

$$\tilde{X}_t(\gamma) = \sum_{i=1}^{m+k} \gamma_i \; X_{t-i}$$

This predictor is improper because it uses a different hypothesis class (higher-order AR) with a different comparator class (ARMA). Furthermore, Anava et al. show that applying the Online Newton Step (ONS) algorithm (as introduced in Section I-B) to the Surrogate AR predictor guarantees logarithmic regret of order $\mathcal{O}(\log T)$ against the best AR(m+k) predictor when applied to convex or exp-concave loss functions $\mathcal{L}(X_t, \tilde{X}_t)$. To prove that this surrogate method is actually beneficial, it is proved that the AR(m+k) approaches the ARMA(p, q) model with a sublinear regret of order $\mathcal{O}(\log^2 T)$ for squared loss and that the average loss suffered by the algorithms based on the improper learning asymptotically converges to the average loss of the best ARMA prediction. To underscore this result, the paper presents benchmarks of the online methods (ARMA-ONS, ARMA-OGD) in comparison to the proper ARMA-RLS learning algorithm and the Yule-Walker estimator on synthetic and real-time series prediction scenarios. As the results show, from the surrogate AR-based predictors that compete against ARMA models, the ARMA-ONS beats all other approaches in 4/4 synthetic scenarios and exhibits notably lower cumulative loss during regime changes see Fig. 4 (Left)), while only being slightly outperformed in the S&P 500 scenario by the Yule-Walker method (see Fig. 4, Right)

C. Application-driven problem reformulation: ternary classification

To summarize, models like ARMA-ONS and ARMA-OGD can be used successfully for forecasting certain time series in a regressive fashion, but despite this, the ARMA / ARIMA framework is still fundamentally challenged by the non-stationary nature of market price dynamics, as noted by Amjad and Shah in [22]. In their paper centered around algorithmic Bitcoin[1] trading, they propose a different approach to time series forecasting. Specifically, they show that although the price of Bitcoin

[1]Bitcoin is a cryptocurrency based on decentralized digital ledger (blockchain) technology

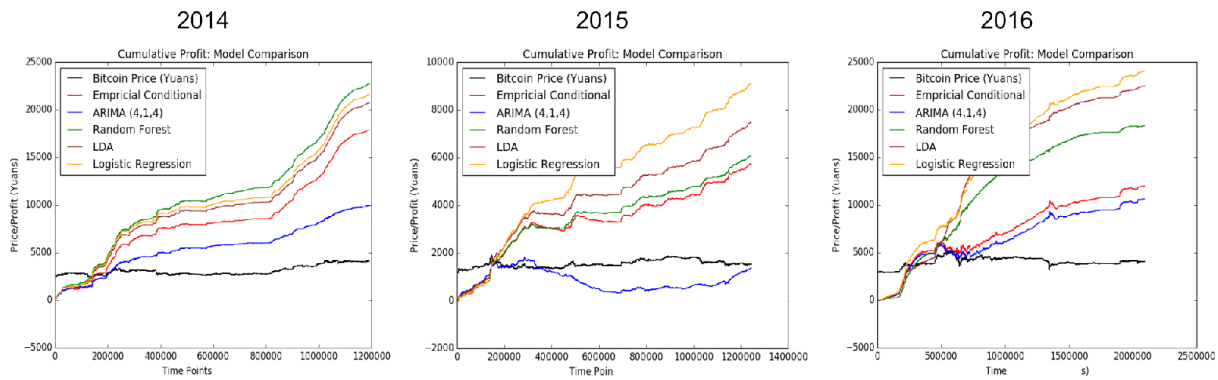

Fig. 5. Cumulative return on investment for different trading strategies over the validation period. The classification-based methods (Random Forest, Logistic Regression, EC) significantly outperform the classical ARIMA model and the baseline "Buy and Hold" strategy [22] (original figure quality is limited).

does not exhibit stationarity - meaning the time series' underlying statistical properties like mean, variance, and covariance change over time - the first-differences series of the prices does in fact exhibit stationarity. It is further shown that Bitcoin's data is also mixing, meaning that the price at time t y_t is primarily dependent on recent price data $y_{t-1}, y_{t-2}, \ldots$ as the influence of the prices decays over time, which makes certain forecasting models feasible.

They further introduce and justify a probabilistic framework for the forecasting, which relies not on regression modeling like the classical ARIMA / ARMA approaches but instead on a reduced forecasting problem, which is basically a ternary classification: the model only needs to calculate probabilities for the three discrete classes of the price change: Up $(+1)$, Down (-1) or No Change (0). Formally, let $y_t = p_t - p_{t-1}$ be the price change at time t. It is assumed that the conditional distribution of the discrete state $x_t \in \{-1, 0, 1\}$ is only dependent on the recent history of the last $d \geq 1$ values. The probability vector for time t is thus defined as $Q_t = (P(x_t = -1), P(x_t = 0), P(x_t = 1))^\top$, being generated by the continuous forecast function $Q_t = F(P(y_{t-d}, \cdots, y_{t-1}))$ with $Q_t : \mathbb{R}^d \to [0, 1]^3$. This definition subsequently allows the model to describe the confidence of each prediction $\hat{x}[t] \in \{-1, 0, 1\}$ at the time $t-1$, which is useful for filtering out low-confidence predictions with a tunable prediction threshold γ, making the model default to a No Change (0) prediction instead. Hence, the algorithm's decision at time t (assuming a long position) can be expressed as

$$D_t = \begin{cases} \text{Sell} & \text{if } P(x_t = -1) \geq \gamma \\ \text{Buy} & \text{if } P(x_t = 1) \geq \gamma \\ \text{Hold} & \text{else} \end{cases}$$

For the function F, Amjad and Shah defined and trained several models, including the classical ARIMA model, as well as classification algorithms like Random Forests, Linear Discriminant Analysis (LDA) and Logistic Regression. Additionally, they implemented an algorithm estimating the empirical conditional (EC) distribution of the data. The training was conducted using features derived from recent price changes extracted from 5-second time intervals over the course of multiple weeks.

The paper compares the performance of the models on different validation sets of multiple weeks of forecasting, after training periods of around one month (see Fig. 5). The results show strong differences in the cumulative profit performance of the models: While the classification-based models achieved consistent profitability in all trials, showing strong 4x and 6x returns on the investment, the ARIMA model's forecasts were prone to larger errors, resulting in cumulative losses or lower profitability when compared to the other methods. Additionally, the ARIMA model underperformed in another experiment, where eight-month-old training data was used for the forecasts. While the other models were similarly profitable, achieving returns between 2x and 4x, the ARIMA model could not handle the older data, which resulted in a negative return multiple of -3.1, as reported in Table 2 of [22].

Unlike the previous regret minimization approach of Anava et al., these classification models were trained offline on historic price data and applied further without online updates. The paper's empirical analysis demonstrates that problem reformulation alone may yield strong performance when compared with a recursively updating online-applied ARIMA model.

To summarize, Amjad and Shah demonstrate that using a classification approach for time series forecasting can result in satisfactory performance on real market data while outperforming classical regression models. This suggests that a careful analysis of the underlying time series properties and a deduction of a specific mode of application (e.g., reframing the problem as a ternary classification task with batch-trained predictors applied in real-time) can prove very beneficial for the forecasting performance. These ideas can be seen as motivation for the development of task-specific proper online learning algorithms for financial time series forecasting.

TABLE I
PREQUENTIAL NEXT-STEP TRADING PERFORMANCE ON SPY (DAILY OPEN PRICES).

Scenario	Model	Trade Acc.	Sharpe	Coverage	Trades
2008 crisis (2006–2010)	Online SNARIMAX	0.347	0.89	0.73	918
	Batch ARIMA	0.320	-0.06	0.74	935
	Ternary Classifier	0.305	-0.50	0.57	714
Tranquil (2015–2018)	Online SNARIMAX	0.200	0.49	0.89	889
	Batch ARIMA	0.192	0.39	0.61	610
	Ternary Classifier	0.262	0.04	0.34	343
COVID-19 (2019–2022)	Online SNARIMAX	0.307	-0.23	0.81	817
	Batch ARIMA	0.287	-0.48	0.70	701
	Ternary Classifier	0.313	-0.09	0.53	530

Trade accuracy is conditional on non-zero signals.
Sharpe is computed on the full opportunity return series, including zeros when flat.

V. COMPARISON OF ONLINE LEARNING MODELS

We evaluate the efficacy of true OL models in financial forecasting by extending the framework of Amjad & Shah's (2017) Bitcoin trading paper [22], which demonstrated the feasibility and potential of ternary classification for trading on streaming price data. While their work relied on batch-retrained models emulating online decision-making, we implement true online learning models using the River library in Python. We compare three distinct approaches:

*1) **Online SNARIMAX**:* A generalized regression model that is incrementally updated, minimizing regret as new observations are made.

*2) **Online Ternary Classifier**:* A proper online version of the Amjad & Shah strategy (forecasting Up/Down/No change) with a confidence threshold $\gamma = 0.55$. The online classification is realized using River's SoftmaxRegression model.

*3) **Batch ARIMA**:* Chosen as a baseline for comparison that is refitted periodically (every 60 trading days) rather than updated continuously.

All models are evaluated on daily S&P 500 open prices while adhering to a strict next-step prediction protocol: at each time t, the model places a prediction for $t + 1$, which is recorded, and only then is the model shown the true value at t. In order to ensure compatibility between the three models, the regression forecasts are converted into ternary trading signals. For this, we introduce a dead-zone threshold of $\pm 0.5\%$ around zero to map forecasted price changes within this band to the neutral class (no trade), avoiding acting on low-magnitude price changes dominated by noise. We evaluate performance across three historic scenarios for the S&P 500: the 2008 crisis (2006–2010), a tranquil phase of relative market stability (2015–2018), and the COVID-19 crisis (2019–2022), in order to test the models' performance during regime shifts and calm phases with clearer trends.

A. Results

Across the three regimes, Online SNARIMAX achieves the strongest risk-adjusted performance, especially during the 2008 financial crisis, where it is the only approach with a positive Sharpe (0.89 vs. -0.06 for periodically refit ARIMA and -0.50 for the online ternary classifier). During the tranquil period (2015–2018), all models perform with positive Sharpe ratios, but Online SNARIMAX remains the best (0.49), closely followed by Batch ARIMA (0.39), while the ternary classifier has flat returns (0.04). During the COVID-19 crisis (2019–2022), the Sharp ratios of all approaches turn negative, but the ternary classifier degrades the least (-0.09), which is consistent with its lower market exposure (0.53 vs. 0.81 for Online SNARIMAX). Overall, trade accuracy is below 0.5 for all methods, indicating that the performance differences boil down to exposure control and asymmetric payoffs rather than frequent correct predictions.

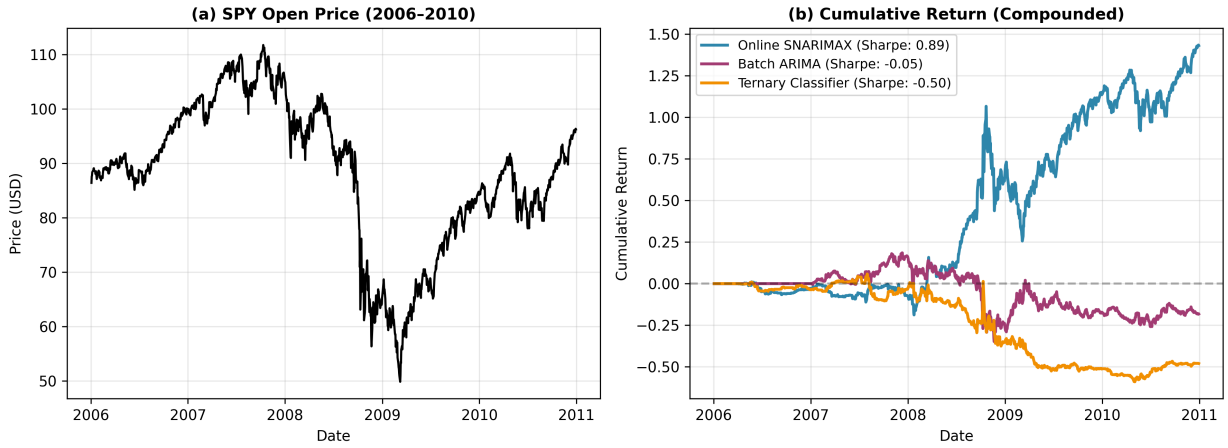

Fig. 6. (a) SPY daily open price during the 2008 crisis window (2006–2010). (b) Compounded cumulative return from prequential next-day trading on SPY for Online SNARIMAX, Batch ARIMA, and an online ternary Up/Down/Flat classifier; regression forecasts are mapped to signals via a $\pm 0.5\%$ dead-zone, and Sharpe is computed on opportunity returns (zeros when flat).

VI. Conclusion

This seminar paper examined the paradigm of online learning with the goal of motivating the use of online learning algorithms and subsequently providing a broad overview of algorithms used for classification and regression problems.

Based on the findings of [16], the survey of practical online learning methods for classification problems showed that the best performance in Task-, Domain- and Class-incremental learning was achieved by a combined use of Deep Generative Replay [4] and knowledge distillation methods [15]. Although the reviewed methods allow for the extension of a classification model to include new class types, this paper did not discuss how online classification can be applied to data streams whose distributions change to both include new classes and eventually exclude old classes. In this case, a certain degree of 'forgetting' would be advantageous to free up new parts of the model parameters for new tasks.

Regarding time series forecasting, our paper highlighted the limitations of classical autoregressive models (ARMA / ARIMA) when applied to non-stationary real-world (market) data. An analysis of [7] revealed that these limitations can be mitigated through *improper learning*, specifically by decoupling the prediction model from the learner class and optimizing a surrogate predictor with the Online Newton Step (ONS) algorithm to achieve logarithmic regret. Further, the consideration of [22] revealed for financial markets that high online forecasting performance can be achieved by departing from the notion of strict regression and by transforming the problem into a discrete *classification task* on price movements, with models trained offline on historical windows and then applied in real time. This flexible approach was shown to improve forecasting accuracy greatly over the ARIMA model. Extending this idea, our empirical model comparison on SPY daily open prices shows that a true online regression model (Online SNARIMAX) achieves the strongest risk-adjusted performance in the 2008 crisis window (Sharpe 0.89), while the more conservative online ternary classifier attains the smallest performance degradation during COVID-19 (Sharpe -0.09) due to lower market exposure. In the experiment, overall trade accuracy remains below 0.5 for all approaches, suggesting that exposure control dominates the trading performance in this setting.

In summary, we note that online time series forecasting appears to be very problem-specific: success often requires selecting models carefully, while sometimes even rethinking the problem formulation itself, moving away from classical regression solutions in favor of problem-specific, empirically motivated approaches.

We acknowledge certain limitations in this seminar paper. First, being a survey, this paper highlights selected problems and algorithms and is not an exhaustive overview of all research in online learning. Second, our empirical model comparison is intentionally kept simple and is restricted to daily S&P 500 data, relying on fixed hyperparameters, such as the dead-zone threshold. As such, the trading performance observed may not generalize to other asset classes and higher trading frequency. Also, for simplicity, we did not include transaction costs, which could affect the comparison.

Future work should address these constraints by modeling realistic market conditions, such as slippage and latency, and by expanding the evaluation to more asset classes. We believe that the discussed modeling approaches can have broader applications, which can be the basis of further scientific study. Finally, for continual classification, future research could explore mechanisms for selective forgetting, enabling models to discard obsolete classes while retaining transferable knowledge.

References

[1] MathWorks, *Incremental learning overview*, https://www.mathworks.com/help/stats/incremental-learning-overview.html, Accessed: 2025-10-23, 2025.

[2] Y. Avneon. "Online incremental learning (oil) system - genuine continuous ml." Conference presentation, YouTube video, accessed: 2025-10-26, AI Summit at Reichman University. (2025), [Online]. Available: https://www.youtube.com/watch?v=Xfayd_uRDkk.

[3] *Machine learning for data streams: with practical examples in MOA* (Adaptive computation and machine learning), en. Cambridge, Mass: The MIT press, 2017, ISBN: 978-0-262-03779-2.

[4] H. Shin, J. K. Lee, J. Kim, and J. Kim, "Continual Learning with Deep Generative Replay," en,

[5] N. Y. Masse, G. D. Grant, and D. J. Freedman, "Alleviating catastrophic forgetting using context-dependent gating and synaptic stabilization," en, *Proceedings of the National Academy of Sciences*, vol. 115, no. 44, Oct. 2018, arXiv:1802.01569 [cs], ISSN: 0027-8424, 1091-6490. DOI: 10.1073/pnas.1803839115. [Online]. Available: http://arxiv.org/abs/1802.01569 (visited on 10/17/2025).

[6] S. Shalev-Shwartz, "Online Learning and Online Convex Optimization," en, *Foundations and Trends® in Machine Learning*, vol. 4, no. 2, pp. 107–194, 2011, ISSN: 1935-8237, 1935-8245. DOI: 10.1561/2200000018. [Online]. Available: http://www.nowpublishers.com/article/Details/MAL-018 (visited on 11/02/2025).

[7] O. Anava, E. Hazan, S. Mannor, and O. Shamir, *Online Learning for Time Series Prediction*, en, arXiv:1302.6927 [cs], Feb. 2013. DOI: 10.48550/arXiv.1302.6927. [Online]. Available: http://arxiv.org/abs/1302.6927 (visited on 10/16/2025).

[8] F. Orabona, *A Modern Introduction to Online Learning*, en, arXiv:1912.13213 [cs], May 2025. DOI: 10.48550/arXiv.1912.13213. [Online]. Available: http://arxiv.org/abs/1912.13213 (visited on 11/02/2025).

[9] S. P. Boyd and L. Vandenberghe, *Convex optimization*, en, Version 29. Cambridge New York Melbourne New Delhi Singapore: Cambridge University Press, 2023, ISBN: 978-0-521-83378-3.

[10] E. Hazan, *Introduction to Online Convex Optimization*, en, arXiv:1909.05207 [cs], Aug. 2023. DOI: 10.48550/arXiv.1909.05207. [Online]. Available: http://arxiv.org/abs/1909.05207 (visited on 12/01/2025).

[11] O. Tasar, Y. Tarabalka, and P. Alliez, "Incremental Learning for Semantic Segmentation of Large-Scale Remote Sensing Data," en, *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 12, no. 9, pp. 3524–3537, Sep. 2019, ISSN: 1939-1404, 2151-1535. DOI: 10.1109/JSTARS.2019.2925416. [Online]. Available: https://ieeexplore.ieee.org/document/8844712/ (visited on 10/24/2025).

[12] F. Zenke, B. Poole, and S. Ganguli, *Continual Learning Through Synaptic Intelligence*, en, arXiv:1703.04200 [cs], Jun. 2017. DOI: 10.48550/arXiv.1703.04200. [Online]. Available: http://arxiv.org/abs/1703.04200 (visited on 11/26/2025).

[13] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, *et al.*, "Overcoming catastrophic forgetting in neural networks," en, *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, Mar. 2017, arXiv:1612.00796 [cs], ISSN: 0027-8424, 1091-6490. DOI: 10.1073/pnas.1611835114. [Online]. Available: http://arxiv.org/abs/1612.00796 (visited on 11/26/2025).

[14] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, *iCaRL: Incremental Classifier and Representation Learning*, en, arXiv:1611.07725 [cs], Apr. 2017. DOI: 10.48550/arXiv.1611.07725. [Online]. Available: http://arxiv.org/abs/1611.07725 (visited on 10/19/2025).

[15] Z. Li and D. Hoiem, *Learning without Forgetting*, en, arXiv:1606.09282 [cs], Feb. 2017. DOI: 10.48550/arXiv.1606.09282. [Online]. Available: http://arxiv.org/abs/1606.09282 (visited on 10/17/2025).

[16] G. M. v. d. Ven and A. S. Tolias, *Three scenarios for continual learning*, en, arXiv:1904.07734 [cs], Apr. 2019. DOI: 10.48550/arXiv.1904.07734. [Online]. Available: http://arxiv.org/abs/1904.07734 (visited on 10/16/2025).

[17] C. Zeno, I. Golan, E. Hoffer, and D. Soudry, *Task Agnostic Continual Learning Using Online Variational Bayes*, en, arXiv:1803.10123 [stat], Feb. 2019. DOI: 10.48550/arXiv.1803.10123. [Online]. Available: http://arxiv.org/abs/1803.10123 (visited on 10/17/2025).

[18] D. Z. für Luft- und Raumfahrt (DLR), *Big data from space – seeking solutions for the flood of data from space*, DLR News Release, Accessed: 2025-11-10, Feb. 2019. [Online]. Available: https://www.dlr.de/en/latest/news/2019/01/20190220_big-data-from-space.

[19] E. G. Tajeuna, M. Bouguessa, and S. Wang, "Modeling Regime Shifts in Multiple Time Series," en, *ACM Transactions on Knowledge Discovery from Data*, vol. 17, no. 8, pp. 1–31, Sep. 2023, ISSN: 1556-4681, 1556-472X. DOI: 10.1145/3592857. [Online]. Available: https://dl.acm.org/doi/10.1145/3592857 (visited on 11/16/2025).

[20] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time series analysis: forecasting and control* (Wiley series in probability and statistics), eng, Fifth edition. Hoboken, New Jersey: Wiley, 2016, ISBN: 978-1-118-67502-1 978-1-118-67492-5.

[21] C. Liu, S. C. Hoi, P. Zhao, and J. Sun, "Online ARIMA Algorithms for Time Series Prediction," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, Feb. 2016, ISSN: 2374-3468, 2159-5399. DOI: 10.1609/aaai.v30i1.10257. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/10257 (visited on 11/16/2025).

[22] M. Amjad and D. Shah, "Trading Bitcoin and Online Time Series Prediction," en, in *Proceedings of the Time Series Workshop at NIPS 2016*, ISSN: 1938-7228, PMLR, Feb. 2017, pp. 1–15. [Online]. Available: https://proceedings.mlr.press/v55/amjad16.html (visited on 12/02/2025).